
PROGRAMMING IN THE LARGE (CSSE2002)
ASSIGNMENT 2 — SEMESTER 2, 2026
SCHOOL OF EECS
THE UNIVERSITY OF QUEENSLAND

Due October 21st 13:00 AEST

*You can, if you want, rewrite forever.
— Neil Simon*

DO NOT DISTRIBUTE.

Version 1.0.0

Overview This assignment delivers experience working with an existing software system. You are given a small extension to the FIREWALL program. The extension has introduced errors and is poorly written. Fortunately, a suite of JUnit tests accompanies the extension.

You will be evaluated on your ability to:

- identify and fix bugs in provided code,
- develop comprehensive JUnit tests that identify mutations,
- meaningfully refactor provided software to improve its quality, and
- your ability to modify/extend your implementation in a time-constrained environment.

Task FIREWALL was a huge success! Everyone wants to play! However, some users felt it was too easy. You decide to make it more interesting by introducing new tower and enemy types into the game. The course staff have tried to add some towers and enemies, but have made a mess of it. Your task is to fix their mistakes, develop comprehensive JUnit tests for the new features, and refactor the existing implementation to improve its quality.

Plagiarism As per assignment 1.

Generative Artificial Intelligence As per assignment 1.

Interviews As per assignment 1.

Practical Demonstration In week 13, you will be required to attend your allocated practical session. These practical demonstrations will be the same process as for assignment 1, refer to the assignment one task sheet for details. If you need to reschedule, you must attend the session on Monday the 2nd of November by filling out the form:

<https://uqrl.au/rl57u>

If you cannot attend that session, you will not receive any marks for the assignment. To give yourself the best chance of making the session, ensure that you do not reschedule unless absolutely necessary.

Software Design

In contrast to assignment 1, you will not be given specification at the method level. Rather, you are provided a high-level specification and accompanying system tests. The rest of the implementation design is up to you. You should use the software design principles (such as coupling, cohesion, information hiding, SOLID, etc.) that are taught in class to help your design. The design of your software is apart of the assessment. Please be aware that:

1. Discussing the design of your classes in detail with your peers may constitute collusion. Please discuss general design principles (cohesion, coupling, etc) but avoid discussing your specific approach to this assignment or how general design principles may apply to the assignment.
2. Course staff will provide minimal assistance with design questions to avoid influencing your approach. You are encouraged to ask general software design questions.

Bug Fixing To begin bug fixing, start with the smallest failing test case — do not begin with a complex test as there are multiple bugs that intersect. Develop some theories about what may be wrong with the implementation, play testing the software may be helpful. You may find it helpful to construct some smaller test scenarios. There are 3 bugs. The bugs are identified by the system tests. You will need to have a clear understanding about what the bug is before attempting to fix it. You may find manually writing the test cases out (i.e. drawing the test scenario) helpful.

Version Control You are encouraged to use version control to track your progress through the assignment. You should aim to use descriptive commit messages as discussed in lectures.

Documentation You will submit a short **2 page maximum, 10pt minimum** written summary of the refactorings you have made and the brief justification for doing so. This should not be a long tale, it should be a concise description that directs us to the refactorings you're most proud of.

System Tests The provided code for this assignment includes JUnit system tests. These are system tests, not unit tests. The goal is for you to have a high degree of flexibility in how you implement your design while still passing these tests. Once you have settled on a design that you're happy with, you will need to write proper unit tests for your classes.

The system tests exist within `test/towers/simulations`. You *must not* modify these tests or anything in `test/test_utils`. Each test class simulates a scenario and inspects the state of the scenario in its test methods.

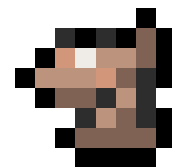
VORTEX TOWER

A vortex tower is a different type of tower that 'fires' a vortex (rather than a bomb) every 1800 ticks (rather than every 200 ticks). A vortex does not move, it stays at the tower spawn location. When a vortex is fired it will kill every enemy within a 200 pixel radius.



HORSE

A horse has similar behaviour to a knight except that it moves twice as fast.



REFACTORING

Your refactoring should only modify code within the following packages/classes (as these are what is marked): `towers.buildings`, `towers.enemies`. You may create additional public classes/methods within these packages to improve the overall design as long as the system tests continue to pass. Tips:

- Ensure that you are continuously running the system tests to ensure that a bug has not been introduced.
- If a section of code is initially difficult to understand, you can likely refactor it for improved comprehension once you understand it.
- Is everything actually used? Are there unused classes or methods?
- Has functionality been appropriately documented? Are there implicit preconditions, postconditions, or invariants that could be made explicit?
- Consider what parts of the game is likely to be changed — how can you make changes easy?
- Consider what parts of the game are likely to be reused if the game is extended — how can you make it easy to reuse those components?

REFACTORING DOCUMENTATION

Your assignment must include a **2 page maximum, 10pt minimum** `refactorings.pdf` file which explains the refactorings that you have made. A **brief** justification for each refactoring choice with respect to the topics covered in the course or personal experience/preference should be included.

This document will be read by course staff while evaluating your assignment against the Readability and Design rubrics. Highlight areas where your assignment meets these criteria.

TASKS

1. Download the provided assignment code:

<https://csse2002.uqcloud.net/assessment/2026s2a2/a2-provided.zip>

2. Open the project.
 - Import the project into IntelliJ.
 - Ensure that the project compiles and runs (including running JUnit tests).
3. Identify and fix the errors/bugs in the provided code.
 - The code contains bugs, causing the provided JUnit tests to fail.
 - You must fix all errors in the code to implement the specification and pass all the tests.
 - You must not change the provided tests in any way.
4. Refactor the provided code.
 - The provided implementation of horses and vertex towers are poorly implemented.
 - You must refactor the provided code to improve its quality.
 - You must not modify the behaviour of the original game while refactoring (i.e., system tests must remain passing).
5. Unit test your implementation.
 - Once you are happy with the public interface of a class, write JUnit tests for the class.
 - Your JUnit tests must be created under `test/towers/enemies`.
 - You must test all classes in `towers.enemies`.
6. Summarize your refactorings in a **2 page maximum** document named `refactorings.pdf`.

MARKING

The assignment will be marked out of 100. The marks will be divided into categories: bug fixes (B), test quality (T), code quality (Q), and style (S).

	Weight	Description
B	10	The errors in the provided code have been fixed.
T	35	The new features of the game are well tested.
Q	45	The new components have good code quality and the provided code has been refactored to improve its quality.
S	10	Code style conforms to course style guides.

The overall assignment mark is defined as

$$A_2 = (10 \times B) + (35 \times T) + (45 \times Q) + (10 \times S)$$

Bug Fixes The provided code includes JUnit tests that fail, indicating an incorrect implementation. You will be awarded marks for modifying the implementation such that the provided JUnit tests pass.

Your mark is based on the number of bugs you fix. The number of bugs you fix is determined by the number of unit tests you pass. For example, assume that the project has 40 unit tests and when given to you, 10 pass. After you have fixed the bugs, 25 tests pass. Then you have fixed 15 out of 30 bugs.¹ Your mark is then

$$\frac{25 - 10}{30} = \frac{15}{30} = 0.5 \text{ (50\%)}$$

In general, let p_0 and f_0 be the number of unit tests that pass and fail in the provided code respectively. If p is the number of unit tests that pass when you submit, then your mark is

$$B = \max\left(\frac{p - p_0}{f_0}, 0\right)$$

Test Quality Your JUnit tests will be marked on their coverage and ability to identify mutations (likely bugs) to your implementation code. For example, imagine your implementation has the following:

```
int add(int x, int y) {  
    return x + y ;  
}
```

We may *mutate* your implementation to change $x + y$ to $x - y$ and then ensure that at least one of your unit tests identifies this mutation (by failing). You should ensure that all your program statements are useful (i.e. contribute to the goal of the method such that changing them would make the method incorrect) and that their behaviour is covered by your JUnit tests.

Gradescope will provide you with some general feedback on test quality.

Code Quality The code quality of new features and refactored existing features will be manually marked by course staff.

¹You do not get any marks for the unit tests that pass before you start and you cannot end up with a negative mark. If your modifications cause a test that originally passed to fail, that no longer counts as a pass.

To do well in this category of the marking criteria, you should consider the software design topics covered in this course. For example, consider the cohesion and coupling of your classes. Ensure that all classes appropriately document their invariants and pre/post-conditions. Consider whether SOLID principles can be applied to your software. The submitted code will be checked against the justification report when marking.

An implementation with high code quality is one that is readable, understandable, maintainable, and extensible. The rubric on the following page details the criteria your implementation will be marked against. Ensure that you read the criteria prior to starting your implementation and read it again close to submission to ensure you meet the criteria.

Code Style The Code Style category is marked starting with a mark of 10. Every occurrence of a style violation in your solution, as detected by *Checkstyle* using the course-provided configuration², results in a 1 mark deduction, down to a minimum of 0. For example, if your code has 2 checkstyle violations, then your mark for code quality is 8. Note that multiple style violations of the same type will each result in a 1 mark deduction.

$$S = \frac{\max(0, 10 - v)}{10}$$

where v is the number of style violations in your submission.

Note: There is a plug-in available for *IntelliJ* which will highlight style violations in your code. Instructions for installing this plug-in are available in the Java Programming Style Guide on the course website. If you correctly use the plug-in and follow the style requirements, it should be relatively straightforward to get high marks for this section.

ELECTRONIC MARKING

Marking will be carried out automatically in a Linux environment. The environment will not be running Windows, and neither IntelliJ nor Eclipse (or any other IDE) will be involved. TemurinJDK 21 with the JUnit 4 library will be used to compile and execute your code and tests. When uploading your assignment to Gradescope, ensure that Gradescope says that your submission was compiled successfully.

Your code must compile.

If your submission does not compile, you will receive zero marks.

²The latest version of the course *Checkstyle* configuration can be found at <http://csse2002.uqcloud.net/checkstyle.xml>. See the Style Guide for instructions.

READABILITY (40%)

Criteria	Advanced (100%)	Functional (75%)	Developing (50%)	Little Evidence (25%)	No Evidence (0%)
Method Decomposition (10%)	All functionality has been decomposed into small coherent methods that have singular clear purposes.	Most functionality has been decomposed into small coherent methods, however, some methods attempt to take responsibility for too much functionality.	The functionality of some methods is not clear as they are either extraneous or perform multiple tasks.	Methods are occasionally decomposed appropriately, however, on the whole most methods perform too many tasks.	Almost no attempt has been made to decompose methods.
Descriptive Naming (10%)	All classes, members, and local variables have clear names that clarify their purpose.	Most classes, members, and local variables have clear names that clarify their purpose.	Some classes, members, or local variables are named poorly and harm the readability of the code.	Minimal attempts have been made to create names that are clarifying.	Almost no attempt has been made to create descriptive names for identifiers.
Documentation (10%)	All classes and public members have clear Javadoc comments that explain how to utilize the classes and members. Documentation may include usage examples.	All classes and public members have Javadoc that attempts to explain how classes and members should be utilized. However, the documentation does not make the purpose and/or expected usage of the software obvious.	All classes and public members have Javadoc comments but they occasionally do not help to explain how to utilize the class in a meaningful way.	Not all classes and public members have Javadoc comments or documentation is not helpful.	Minimal evidence or no evidence of Javadoc documentation throughout the submission.
Program Structure (10%)	The structure of code within methods is clear. Vertical spacing is utilized to separate any logical blocks of code. The control structures are suitable for the task. Any complex blocks or lines of code have been minimized and are appropriately documented.	The structure of code within methods is appropriate but not clear. Vertical spacing is mostly utilized to separate any logical blocks of code. Control structures are somewhat convoluted but mostly appropriate. Any complex blocks or lines of code have been minimized or are appropriately documented.	The structure of code within methods occasionally makes it difficult to understand the intention of the code. An attempt has been made to structure the code but falls short of being well structured.	The structure of code within methods makes the intention of the code or functionality of the method difficult to understand.	The structure of code within methods is poorly structured and hard to read.

DESIGN (60%)

Criteria	Advanced (100%)	Functional (75%)	Developing (50%)	Little Evidence (25%)	No Evidence (0%)
Information Hiding (15%)	All classes hide and protect their internal representation. References to internal state is protected by appropriate copying. An appropriate abstraction has been created around the internal representation that would make it feasible to later change implementations.	All classes attempt to hide and protect their internal representation, however, some leakage of internal state may occur. Consideration has been made towards creating an appropriate data abstraction.	Most classes have attempted to protect their internal representation. Some methods that should be private are public, damaging the data abstraction.	Minimal attempts have been made to protect the internal representation of classes. Methods that should be private are public or member variables have been made public.	Member variables/methods are public in a way that encourages tight coupling. No or minimal evidence of data abstraction.
Dependency Inversion (15%)	The logic of the program is inverted in a way that makes most classes highly configurable. The domain logic of the program is passed down allowing easy future modification.	It is possible to alter some of the program logic via dependency injection. However, non-trivial changes would mostly still require modification.	An attempt has been made to parameterise classes by abstractions of domain logic. However, the abstractions are not at an appropriate level.	No attempt has been made to enable changing the program logic by dependency inversion. Some attempt has been made to parameterise classes but program logic is rigid.	The software is highly rigid, any alterations of the program logic would require modification of low-level program components.
Cohesion (10%)	All classes are highly cohesive with each serving a clear single-minded purpose.	Most classes are highly cohesive, serving a clear purpose.	There are no “God classes” but not all classes are highly cohesive.	There is at least one “God class” that assumes too many responsibilities.	No/minimal attempt has been made to decompose classes such that each is cohesive.
Polymorphism (10%)	Polymorphism is used to enhance the flexibility of the program. No subclasses duplicate the functionality of the parent. No classes violate the substitution principle.	Polymorphism is used and no subclasses duplicate the functionality of their parent. No classes violate the substitution principle.	Polymorphism is used and some subclasses duplicate the functionality of their parent. No classes violate the substitution principle.	Polymorphism is used but subclasses duplicate some functionality of their parent or violate the substitution principle.	Polymorphism has not been used.
Contract Programming (10%)	Where appropriate, classes include documented class invariants and pre/post-conditions on public members.	Where appropriate, most classes include documented class invariants and pre/post-conditions on public members.	Some classes have documented their class invariants and pre/post-conditions on public members.	Some classes lack important documentation of their class invariants and/or pre/post-conditions on public members.	Minimal or no attempt has been made to document the invariants/pre/post-conditions on public members.

SUBMISSION

Submission is via Gradescope as with Assignment 1.

Submit your code to Gradescope *early and often*. Gradescope will give you some feedback on your code, but it is not a substitute for testing your code yourself.

You must submit your code and document *before* the deadline. Anything that is submitted after the deadline will **not** be marked (1 nanosecond late is still late). See Assessment Policy.

You may submit your assignment to Gradescope as many times as you wish before the due date. Your last submission made before the due date will be marked.

Code Submission

Your code submission must include the following:

ai/README.md

- A declaration of your AI usage.

src/towers

- All classes you created/modified in the refactor.

test/towers

- Any JUnit tests you wrote.

refactorings.pdf

You should use the `bundle.bat` (Windows) or `bundle.sh` (everything else) to package your code for submission. **Do not** include the provided code outside of these packages. If you create additional packages, include them in the submission. Only submit code you have modified or created.

ASSESSMENT POLICY

Late Submission Late submissions will incur a penalty as specified in the course profile.

Extensions Extensions may be applied for as specified in the course profile.

Remarking If an *administrative error* has been made in the marking of your assignment (e.g. marks were incorrectly added up), please contact the course coordinator (csse2002@uq.edu.au) to request this be fixed.

For all other cases, please refer to the following page for further information:

<http://uqrl.au/rl552>

CHANGE LOG

VERSION: 1.0.0

Any updates to the assignment will be documented here.